

API Security Risk Analysis Report

Future Interns — Cyber Security Track

Task 3 · FUTURE_CS_03

1. Executive Summary

This report presents the findings of a security assessment performed against a publicly hosted REST API (ReqRes.in), used here as a representative test target for practicing API security analysis. The assessment focused on the areas most commonly exploited in real-world API breaches, per the OWASP API Security Top 10: authentication handling, authorization / access control, error handling, data exposure, and rate limiting.

Overall, the tested API demonstrated solid fundamentals — authentication was correctly enforced and error handling did not leak sensitive internal details. Two lower-severity issues were identified: the absence of observable rate limiting, and the exposure of user email addresses in a list-style response. Neither issue is critical, but both are worth remediating to align with security best practices before the API is used in a production or client-facing context.

2. Scope & Methodology

Target: <https://reqres.in> (public REST API used for testing purposes).

Tools used:

- Postman — used to craft and send HTTP requests with custom headers
- Browser DevTools (Network tab) — used to observe live API traffic

Testing approach: manual, black-box style testing of publicly documented endpoints. No automated scanning tools or brute-force attacks were used, in line with responsible use of a shared public test service.

Areas tested:

- Authentication — behavior with and without a valid API key
- Authorization / access control — behavior when requesting different and non-existent resource IDs
- Error handling and information disclosure
- Data exposure in standard responses
- Rate limiting under repeated requests

3. Findings

3.1 Issues Identified

Finding	Risk Level	Business Impact	Recommendation
Missing rate limiting on public endpoints	Medium	Without limits on how many requests a client can send, an attacker could hammer the API to scrape all user data quickly, or attempt brute-force attacks on login-style endpoints without being slowed down or blocked.	Implement rate limiting (e.g., 100 requests/minute per API key or IP) and return HTTP 429 once the limit is exceeded.
User email addresses exposed in list responses	Low	The <code>/api/users</code> endpoint returns full email addresses for every user in the system. If this pattern were used for real customer data, it could expose personal information more broadly than necessary,	Return only the minimum fields needed by the consuming application. Mask or omit emails from list views; expose full details only on a per-user

Finding	Risk Level	Business Impact	Recommendation
		especially to any client with a valid key.	detail request with proper authorization checks.

3.2 Good Security Practices Observed

The following behaviors reflect good security design and are noted for completeness:

- Authentication is enforced: requests without a valid x-api-key header are correctly rejected with HTTP 401 Unauthorized, and are not able to view any user data.
- Error handling is clean: querying a non-existent user ID (e.g., /api/users/23) returns a proper HTTP 404 Not Found with no data or internal server details leaked.
- Error messages are informative but safe: the API explains how to fix an auth error (e.g., "add x-api-key header") without exposing stack traces, internal file paths, or database information.

4. Detailed Test Results

4.1 Authentication Testing

Request: GET /api/users without an x-api-key header.

Result: HTTP 401 Unauthorized. The API returned a clear, structured error explaining that the x-api-key header is required, along with guidance on how to obtain and use a key. No user data was returned.

Assessment: Authentication is correctly enforced at the endpoint level. This prevents anonymous/unauthenticated access to user data.

4.2 Authorization Testing (Object-Level Access Control)

Request 1: GET /api/users/2 (valid, existing user ID) with a valid API key.

Result: HTTP 200 OK. Returned the correct single user record for ID 2 only — no other users' data was included.

Request 2: GET /api/users/23 (a user ID expected not to exist) with a valid API key.

Result: HTTP 404 Not Found, with an empty response body.

Assessment: The API correctly scopes responses to the requested resource and does not leak data for invalid or out-of-range IDs. This is consistent with protection against Broken Object Level Authorization (BOLA), the #1 issue in the OWASP API Security Top 10 — no evidence of BOLA was found in this endpoint.

4.3 Data Exposure Review

The GET /api/users response includes each user's id, email, first_name, last_name, and avatar URL. Email addresses are returned in full plaintext for every user in the listing, without any masking.

Assessment: For a fake/demo API this is expected and low-impact. However, if this same response pattern were applied to a real production user directory, exposing every user's email address to any client holding a valid API key would represent an unnecessary data exposure risk, particularly if the API key can be obtained by low-trust or external parties.

4.4 Rate Limiting Testing

Approach: Ten consecutive GET /api/users requests were sent in rapid succession using Postman.

Result: All ten requests returned HTTP 200 OK. No HTTP 429 (Too Many Requests) or other throttling response was observed.

Assessment: No rate limiting was detected within this small sample. This is a limited test (10 requests) rather than an exhaustive load test, but the complete absence of any throttling signal even at this volume suggests the endpoint may not be protected against automated scraping or brute-force style abuse.

5. Risk Summary

Of the two issues identified, none were rated High or Critical. The missing rate limiting is rated Medium since it could enable automated abuse at scale. The email exposure is rated Low, since ReqRes is explicitly a public test API with fictional data; the same pattern would warrant a higher rating on a production system handling real personal data.

6. Recommendations

- Implement rate limiting (e.g., token-bucket or fixed-window, 60–100 requests/minute per API key) with HTTP 429 responses once exceeded.
- Apply the principle of least data exposure: return only fields the consuming client actually needs, especially in list/bulk endpoints.
- Continue enforcing strict authentication and clean 404 handling, as currently implemented — these should be maintained in any future changes.
- Periodically re-test authorization boundaries (BOLA) whenever new endpoints or user roles are introduced.

7. Conclusion

This assessment did not identify any critical or high-severity vulnerabilities in the tested API. Authentication and authorization controls were implemented correctly and error handling was clean and safe. The two lower-severity findings — lack of observable rate limiting and plaintext email exposure in list responses — are common, low-cost fixes that would further harden the API against automated abuse and unnecessary data exposure.

Prepared as part of the Future Interns Cyber Security Internship — Task 3.